Produces and Consumes Attributes in ASP.NET Core Web API

Back to: ASP.NET Core Web API Tutorials

Produces and Consumes Attributes in ASP.NET Core Web API

In this article, we will discuss the need and use of Produces and Consumes Attributes in an ASP.NET Core Web API application with Real-time examples. Content Negotiation automatically handles most request and response formatting in ASP.NET Core Web API. However, in certain real-world situations, as developers, we need **explicit control** over what media types our API **accepts** and **returns**.

This is where the **[Produces]** and **[Consumes]** attributes come into play. They help us define **strong media-type contracts** between the server and clients, ensuring predictable communication across all platforms.

What is the [Produces] Attribute?

The [Produces] attribute defines **What Format (Media Type)** an action method or controller will **produce in its Response**. The [Produces] attribute tells the framework: “Regardless of what the client requests, send the response in this specific format.”

In other words, [Produces] overrides the **Default Output Formatter Selection** made by ASP.NET Core’s Content Negotiation System. This ensures that every response follows a **Consistent, Predefined Structure and Format**.

Why Do We Need [Produces]?

In real-world API development, consistency is critical. Consider these cases:

- You are building an **internal XML-based API** for a legacy enterprise system that can only consume XML.
- You want all your endpoints to **return JSON**, even if a client mistakenly requests XML.
- You need your Swagger or OpenAPI documentation to explicitly show which **Response Formats** are supported.

The [Produces] attribute gives you this level of control.

How It Works

When applied, [Produces] modifies the **response Content-Type header** and instructs ASP.NET Core to use the corresponding **Output Formatter** (e.g., JSON or XML serializer). If multiple media types are provided, the framework selects the first supported type in the list.

Syntax

[Produces("application/json")]

You can also specify multiple media types:

[Produces("application/json", "application/xml")]

Example 1 – Forcing XML Response

In our EmployeeController, let's say one endpoint must always return XML, regardless of what the client requests.

```
using Microsoft.AspNetCore.Mvc;
using ContentNegotiationDemo.Models;

namespace ContentNegotiationDemo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("xml")]
        [Produces("application/xml")]
        public ActionResult<List<Employee>> GetEmployeesInXml()
        {
            var employees = new List<Employee>()
            {
                new Employee(){ Id = 1001, Name = "Anurag", Gender = "Male", Age
= 28, Department = "IT" },
                new Employee(){ Id = 1002, Name = "Pranaya", Gender = "Male", Age
= 28, Department = "IT" }
            };

            return Ok(employees);
        }
    }
}
```

Explanation

Even if a client sends a request like:

- GET /api/employee/xml

- Accept: application/json

The server will still respond with:

- Content-Type: application/xml

This is because [Produces("application/xml")] instructs ASP.NET Core to use the XML formatter explicitly.

Enable XML Formatter in Program Class:

Please modify the Program class as follows.

```
namespace ContentNegotiationDemo
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            builder.Services.AddControllers()
            .AddXmlSerializerFormatters() //Enable XML Formatter
            .AddJsonOptions(options =>
            {
                // Configure JSON serializer settings if needed
                options.JsonSerializerOptions.PropertyNamingPolicy = null;
            });

            // Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
            builder.Services.AddEndpointsApiExplorer();
            builder.Services.AddSwaggerGen();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (app.Environment.IsDevelopment())
            {
                app.UseSwagger();
                app.UseSwaggerUI();
            }

            app.UseHttpsRedirection();

            app.UseAuthorization();
```

```
            app.MapControllers();

            app.Run();
        }
    }
}
```

Now, test the endpoint, and irrespective of the Accept header value, the server is always going to return the Data in XML format.

Example 2 – Supporting Multiple Response Formats

You can also allow both JSON and XML outputs, but restrict the response to these two only:

```
using Microsoft.AspNetCore.Mvc;
using ContentNegotiationDemo.Models;
namespace ContentNegotiationDemo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpGet("multi")]
        [Produces("application/json", "application/xml")]
        public ActionResult<IEnumerable<Employee>> GetEmployees()
        {
            var employees = new List<Employee>()
            {
                new Employee(){ Id = 1003, Name = "John", Gender = "Male", Age =
30, Department = "Finance" },
                new Employee(){ Id = 1004, Name = "Sita", Gender = "Female", Age
= 26, Department = "HR" }
            };

            return Ok(employees);
        }
    }
}
```

Explanation

- The API endpoint will only produce **JSON** or **XML**, no other format.
- The negotiation happens only between these two options.
- This improves API **predictability** and **documentation clarity**.

What is the [Consumes] Attribute?

The [Consumes] attribute defines **what type of request content** your action method can accept. It controls the **input formatter** that ASP.NET Core uses to deserialize incoming data.

When a request's Content-Type header doesn't match any media type specified in [Consumes], ASP.NET Core automatically returns an **HTTP 415 – Unsupported Media Type** response.

Why Do We Need [Consumes]?

In modern APIs, clients can send data in various formats, JSON, XML, form-data, etc. However, most APIs are designed to handle **One Specific Format** for simplicity. Using [Consumes] helps us:

- Enforce **Input Consistency** across all clients.
- Prevent **Invalid or Unsupported Formats** from being processed.
- Clearly document which input formats your API supports.

How It Works

When a client sends a request, ASP.NET Core checks the **Content-Type** header and matches it with the list specified in **[Consumes]**. If no match is found, the request is rejected before even reaching your action logic.

Syntax

[Consumes("application/json")]

You can also specify multiple media types:

[Consumes("application/json", "application/xml")]

Example 3 – Accepting Only JSON Requests

In EmployeeController, let's say we want to allow new employees to be added only if the request body is in JSON format.

```
using Microsoft.AspNetCore.Mvc;
using ContentNegotiationDemo.Models;
namespace ContentNegotiationDemo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpPost]
        [Consumes("application/json")]
```

```
        public IActionResult AddEmployee([FromBody] Employee employee)
        {
            if (employee == null)
                return BadRequest("Invalid data format.");

            // Logic to save employee
            return CreatedAtAction(nameof(AddEmployee), new { id = employee.Id },
employee);
        }
    }
}
```

Explanation

If a client sends the following request:

- POST /api/employee
- Content-Type: application/xml

The response will be:

- HTTP/1.1 415 Unsupported Media Type

This is because only application/json is allowed as per [Consumes].

Example 4 – Accepting XML Requests for Legacy Systems

If your API integrates with an old HR system that only sends XML data, you can handle XML requests specifically:

```
using Microsoft.AspNetCore.Mvc;
using ContentNegotiationDemo.Models;

namespace ContentNegotiationDemo.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EmployeeController : ControllerBase
    {
        [HttpPost("xml")]
        [Consumes("application/xml")]
        [Produces("application/xml")]
        public IActionResult AddEmployeeXml([FromBody] Employee employee)
        {
            if (employee == null)
                return BadRequest("Invalid data.");
```

```
            return CreatedAtAction(nameof(AddEmployeeXml), new { id = employee.Id
}, employee);
        }
    }
}
```

Explanation

This method:

- Accepts only XML requests.
- Returns responses in XML.
- It is ideal for backward compatibility with legacy applications.

Applying [Produces] and [Consumes] at the Controller Level

Sometimes, you may want every action within a controller to adhere to the same request and response format. You can apply these attributes at the **Controller Level**, and they will automatically apply to all action methods unless overridden individually.

```
using Microsoft.AspNetCore.Mvc;
using ContentNegotiationDemo.Models;

namespace ContentNegotiationDemo.Controllers
{
    namespace ContentNegotiationDemo.Controllers
    {
        // Applying [Produces] and [Consumes] at the Controller Level
        // All actions in this controller will accept and return JSON by default
        [ApiController]
        [Route("api/[controller]")]
        [Produces("application/json")]          // Default response format for
all actions
        [Consumes("application/json")]          // Default request format for all
actions
        public class EmployeeController : ControllerBase
        {
            // In-memory Employee List for demonstration
            private static List<Employee> _employees = new List<Employee>
            {
                new Employee(){ Id = 1001, Name = "Anurag", Gender = "Male", Age
= 28, Department = "IT" },
                new Employee(){ Id = 1002, Name = "Pranaya", Gender = "Male", Age
= 28, Department = "IT" },
```

```
                new Employee(){ Id = 1003, Name = "Priyanka", Gender = "Female",
Age = 26, Department = "HR" }
            };

            // Action 1: Default behavior (inherits [Produces] and [Consumes]
from controller)
            [HttpGet]
            public ActionResult<IEnumerable<Employee>> GetAllEmployees()
            {
                return Ok(_employees);
            }

            // Action 2: Add a new employee - uses JSON input as per controller-
level setting
            [HttpPost]
            public IActionResult AddEmployee([FromBody] Employee employee)
            {
                if (employee == null)
                    return BadRequest("Employee data is missing.");

                employee.Id = _employees.Max(e => e.Id) + 1;
                _employees.Add(employee);

                return CreatedAtAction(nameof(GetEmployeeById), new { id =
employee.Id }, employee);
            }

            // Action 3: Get a specific employee by ID - still inherits JSON
response
            [HttpGet("{id:int}")]
            public ActionResult<Employee> GetEmployeeById(int id)
            {
                var employee = _employees.FirstOrDefault(e => e.Id == id);
                if (employee == null)
                    return NotFound($"Employee with ID {id} not found.");

                return Ok(employee);
            }

            // Action 4: Override [Produces] and [Consumes] to use XML for
specific endpoint
            [HttpPost("add-xml")]
            [Consumes("application/xml")]          // Accept only XML request
            [Produces("application/xml")]          // Respond only in XML
            public IActionResult AddEmployeeXml([FromBody] Employee employee)
            {
                if (employee == null)
```

```
                return BadRequest("Invalid XML data.");

            employee.Id = _employees.Max(e => e.Id) + 1;
            _employees.Add(employee);

            return CreatedAtAction(nameof(GetEmployeeByIdXml), new { id =
employee.Id }, employee);
        }

        // Action 5: Return data in XML format - overrides controller-level
[Produces]
        [HttpGet("xml/{id:int}")]
        [Produces("application/xml")]          // Respond in XML regardless
of Accept header
        public ActionResult<Employee> GetEmployeeByIdXml(int id)
        {
            var employee = _employees.FirstOrDefault(e => e.Id == id);
            if (employee == null)
                return NotFound($"Employee with ID {id} not found.");

            return Ok(employee);
        }

        // Action 6: Accept multipart/form-data for file upload - overrides
controller-level [Consumes]
        [HttpPost("upload")]
        [Consumes("multipart/form-data")]      // Accept only form-data
        [Produces("application/json")]         // Still respond in JSON
        public IActionResult UploadFile(IFormFile file)
        {
            if (file == null || file.Length == 0)
                return BadRequest("No file uploaded.");

            // File saving logic can go here (for demo, we only return
metadata)
            return Ok(new
            {
                FileName = file.FileName,
                Size = file.Length,
                Message = "File uploaded successfully."
            });
        }
    }
}
```

Controller-Level [Produces] and [Consumes]

At the top of the controller:

- [Produces("application/json")]
- [Consumes("application/json")]

These attributes set the **default behavior** for every action method:

- All responses are serialized to **JSON**.
- All requests must be in **JSON** format.
- Clients sending XML or form data will receive **415 Unsupported Media Type**.

This ensures **consistent API behavior** across the entire controller.

Overriding at Action Level

ASP.NET Core allows overriding controller-level settings for **specific endpoints**:

AddEmployeeXml()

- Uses [Consumes("application/xml")] and [Produces("application/xml")].
- Accepts XML input and produces XML output, overriding the controller's JSON defaults.
- Ideal for legacy system integration.

GetEmployeeByIdXml()

- Overrides only the [Produces] attribute.
- Response will always be XML, regardless of the client's Accept header.

UploadFile()

- Overrides [Consumes] with "multipart/form-data".
- Accepts file uploads but still returns a JSON response.

Summary:

The [Produces] and [Consumes] attributes are not just about format enforcement — they define **communication contracts** between your API and its consumers.

They ensure:

- Predictable and documented behavior.
- Fewer integration errors.

- Compatibility with legacy or external systems.
- Cleaner and more accurate Swagger/OpenAPI documentation.

In essence:

- **[Produces]** defines how your API **speaks** (the response format).
- **[Consumes]** defines what your API **listens to** (the request format).

Together, they bring **precision**, **consistency**, and **professional robustness** to your ASP.NET Core Web API design.

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.